

Surplus MealGuard AI

References, Summary & Research Stance

Generated: 29 September 2026

1. Project summary

Surplus MealGuard AI is a Smart India Hackathon (SIH 2026) prototype for institutional kitchens (college canteens, messes, enterprise dining). It learns demand from historical records (AI Kitchen Digital Memory, algorithm memory-v1), sets production targets with safety buffers, flags mid-service surplus risk (Surplus-Before-Surplus), closes the day with waste analysis, gates redistribution behind operational eligibility and mandatory human sign-off, matches surplus to NGOs via deterministic scoring (no maps/GPS), and reports estimated financial and CO2e impact with CSV/XLSX/PDF exports and methodology disclaimers.

The README and Demo Guide emphasize honest prototype boundaries: deterministic statistics (not generative/deep learning), operational decision support (not food-safety certification), and labeled estimates for impact—not sensor measurements.

2. References in this project

There is no formal academic bibliography (no .bib file, no DOI/paper list). References in the codebase are documentation, dependencies, configurable factor sources, and internal build plans.

2.1 Primary user-facing documentation

- README.md — SIH context, 15-step demo, features, limitations, stack, repo layout.
- client/src/pages/DemoGuide.tsx — 15-Step SIH Evaluation Guide, credentials, prototype disclaimers.
- docs/controls-audit.md — UI/API no-dead-controls audit checklist.

2.2 Internal implementation plans (engineering specs)

Under docs/superpowers/plans/ (2026-09-29): goals, architecture, and algorithms per build prompt.

- prompt2 — menu, food items, CSV/Excel import.
- prompt3 — memory-v1 forecasting (same-weekday, age-decayed baseline, menu multiplier, confidence).
- prompt4 — production targets, buffers, dashboard.
- prompt5 — live flow, pace/risk, mitigation actions.
- prompt6 — EOD close, waste analysis, eligibility.
- prompt7 — NGO matching and redistribution lifecycle.
- prompt8 — impact factors, analytics, report exports + methodology blocks.
- prompt9-10 — hardening and demo dataset.

2.3 Methodology in core libraries (no external citations)

- kitchen-memory.ts — same-weekday history, weighted baseline, menu multiplier, variance-based confidence.
- surplus-risk.ts — pace from timestamped sales vs meal windows (defaults: Breakfast 07:00–10:30, Lunch 11:30–15:00, Dinner 18:00–21:30).
- waste-analysis.ts — rules over persisted records (demand-overestimate, overproduction, day-pattern, menu-combination).
- eligibility.ts — operational cutoffs; explicitly not a safety certificate.
- matching.ts — weighted criteria on structured/text fields; no distance APIs.
- impact.ts — foodSaved = redistributed + prevented waste vs baseline; cost/CO2 from ImpactFactor rows.

2.4 Configurable sources for impact (demo defaults)

- CO2_PER_KG_FOOD: 2.5 kgCO2e/kg — source: "Seed default - replace with measured factors." (server/src/seed.ts)
- COST_PER_KG_FOOD: 120 INR/kg — same placeholder source.
- Admins may define factors with required source string; reports label cost/CO2 as estimates (server/src/routes/reports.ts).

2.5 Technical / software references

- Server: Express, Prisma/SQLite, Zod, bcryptjs, cors, express-rate-limit, csv-parse, multer, nodemailer, pdfkit, xlsx, @supabase/supabase-js.
- Client: React 18, Vite 5, TypeScript 5, React Router 6, Tailwind 3.
- Import reference templates: server/src/routes/imports.ts (public sample CSV/XLSX).
- Optional Supabase URL in client/src/supabase.ts (env-overridable).

2.6 Demo / evaluation references

- Demo credentials: README + Demo Guide (admin@mealguard.local, manager@mealguard.local, etc.).
- Synthetic history: ~12 weeks / ~600 records with deterministic seeded variation.
- render.yaml — deployment configuration hint.

3. Research & evidence stance

3.1 Not cited in code or docs

- Academic papers, IPCC/WHO/FSSAI documents, SDG official metrics, or national food-waste statistics URLs.
- External ML/LLM forecasting or matching models.
- Map/routing, IoT, vision, or weather APIs (explicitly out of scope in README).

3.2 What the prototype treats as evidence

- Problem framing: SIH-style waste prevention, redistribution, impact reporting.
- Method: explainable rule-based / lightweight time-series for auditability and reproducibility.
- Validation: automated tests (unit, API, e2e) and controls audit—not field studies or published benchmarks.
- Ethics: deterministic statistical AI, human-confirmed eligibility, factor-based impact estimates.

4. Consolidated reference list (in-repo)

- Smart India Hackathon (SIH) 2026 — stated target event.
- Project README — canonical features and limitations.
- Demo Guide (/demo-guide) — evaluation script and integrity notes.
- Eight docs/superpowers/plans/2026-09-29-*.md files — internal design records.
- Open-source stack — server/package.json and client/package.json.
- Default impact factors — seed values with placeholder sources.
- Sample import templates — reference data format for historical records.

This document was generated from repository documentation and source comments only. No application logic was modified.